Support de Cours
Module : Conception par contrats
MR1_GL
Chapitre 1: Introduction

Réalisé par:
Dr. Sakka Rouis Taoufik

1

Qu'est-ce que un système ?

- Un système est un ensemble d'*éléments* interagissant entre eux suivant un certains nombres de principes et de *règles* dans le but de réaliser un *objectif*.
- L'environnement est la partie du monde extérieure au système.
- Un système complexe se caractérise par :

➢ sa dimension, qui nécessite la collaboration de plusieurs personnes

➢ son évolutivité.

Exemple

- l'économie mondiale, le noyau Linux, . . .

2

Qu’est-ce qu’un logiciel ?

Définition

Un logiciel est un ensemble d’entités nécessaires au fonctionnement d’un processus de traitement automatique de l’information.

Parmi ces entités, on trouve par exemple :

- des programmes (en format *code source* ou exécutables) ;
- des documentations d’utilisation ;
- des informations de configuration.

3

Qu’est-ce qu’un logiciel ?

Un logiciel est en général un sous-système d’un système englobant.

- Il peut interagir avec des clients, qui peuvent être :
  - des opérateurs humains (utilisateurs, administrateurs, etc.) ;
  - d’autres logiciels ;
  - des contrôleurs matériels.
- Il réalise une spécification : son comportement vérifie un ensemble de critères qui régissent ses interactions avec son environnement.

4

Qu’est-ce que le génie logiciel ?

Définition (Génie logiciel)

- Le génie logiciel est un domaine des sciences de l’ingénieur dont l’objet d’étude est la conception, la fabrication, et la maintenance des *systèmes informatiques complexes*.

5

Pour quoi le génie logiciel ?

- Le génie logiciel vise à garantir que :
  1) la spécification répond aux besoins réels de ses clients ;
  2) le logiciel respecte bien sa spécification ;
  3) les coûts alloués pour sa réalisation sont respectés ;
  4) les délais de réalisation sont respectés.

6

Comment spécifier un logiciel ?

Répondre à la question

Que doit faire le logiciel ?

La spécification d'un logiciel peut prendre de nombreuses formes.

La *complexité* et les *dimensions* de la spécification peuvent varier énormément en fonction de l'environnement d'utilisation du logiciel et des objectifs auxquels il répond.

7

Spécification informelle

C'est une description des exigences en langue naturelle

Exemple : (Un algorithme de tri)

- *Entrée* un tableau *t*
- *Pré-condition:* il existe une relation d'ordre sur les éléments du tableau
- *Sortie* un tableau *u*
- *Post-condition:* *u* est trié et contient exactement les mêmes éléments que *t*

➔C'est une spécification simple dont la conformité aux objectifs de leurs clients ne fait aucun doute.

8

Spécification formelles

Les approches formelles utilisent des outils mathématiques et des méthodes de preuve pour construire un logiciel correct par construction dont la vérification est automatisée ou assistée.

Exemple (approches formelles)

- Méthodes : méthode B, model-checking, csp de Hoare,…
- Outils et notations : Coq, Z, Atelier B, Why, Frama-C, etc.

Ces méthodes sont utilisées pour développer des logiciels critiques. Elles correspondent au niveau le plus élevé de certification.

➢ e.g. applications de la méthode B pour développer le logiciel embarqué des lignes de métro 14 (1998) et 1 à Paris

9

Spécification semi-formelles

Les approches semi-formelles visent à introduire un langage normalisé pour décrire le logiciel et sa spécification.

- Cependant, la *sémantique du langage de spécification n'est pas formalisée*.
- Bien que ces approches précisent le discours du concepteur si on le compare à celui décrit à l'aide du langage naturel, elles contiennent certaines ambiguïtés et n'offrent aucune garantie sur la qualité des résultats.

Exemple (approches semi-formelles)

- Méthodes : Rationale Unified Process, Merise, . . .
- Outils et notations : UML, AnalyseSI, . . .

Ces méthodes sont utilisées aujourd'hui par l'industrie du logiciel.

10

Comment fabriquer un logiciel de qualité ?

Historiquement, il y a eu une crise de conscience dans les années 70, appelée la crise du logiciel, dû à un tournant décisif : c'est à cette époque que le coût de construction du logiciel est devenu plus important que celui de la construction du matériel.

11

Comment fabriquer un logiciel de qualité ?

Pour répondre à cette crise, on a essayé d'appliquer les méthodes éprouvées de l'ingénieur au domaine du logiciel, pour établir des méthodes fiables sur lesquelles construire une industrie du logiciel.

➔ Il s'agit de se donner un cadre rigoureux pour :

1. Guider le développement du logiciel, de sa conception à sa livraison.
2. Contrôler les coûts, évaluer les risques et respecter les délais.
3. Établir des critères d'évaluation de la qualité d'un logiciel.

12

Comment fabriquer un logiciel de qualité ?

La qualité d'un logiciel dépend principalement des 5 critères suivants :

- Validité (correction, justesse, conformité) : Le logiciel remplit exactement les fonctions définies par le cahier des charges et les spécifications?
- Maintenabilité : Peut-on faire évoluer le logiciel ?
- Robustesse (fiabilité, sureté) : Le logiciel est-il sujet à des dysfonctionnements ?
- Efficacité : Le logiciel fait-il bon usage de ses ressources ?
- Utilisabilité : Est-il facile à utiliser ?

13

Activités du développement de logiciel

Comme pour tout produit manufacturé complexe :

- on **décompose** la production en « **phases** »
- l'ensemble des phases constitue un « **cycle de vie** »
- les phases font apparaître des **activités** clés

14

Activités du développement de logiciel

- analyse des besoins
- spécification
- conception
- programmation
- intégration
- vérification et validation

15

Analyse des besoins

- **But :**
  - → déterminer **ce que doit faire** (et ne pas faire) **le logiciel**
  - → déterminer les **ressources**, les **contraintes**
- **Caractéristiques :**
  - → parler **métier** et non info
  - → entretiens, questionnaires
  - → **observation** de l'existant, **étude** de situations similaires
- **Résultat :** ensemble de documents
  - → **rôle** du système
  - → future **utilisation**
  - → aspects de l'**environnement**
  - → (parfois) un manuel d'utilisation préliminaire

16

Spécification

- **But :**
  - → établir une 1ère description du futur système
  - → consigner dans un document qui fait référence
- **Données :**
  - → résultats de l'analyse des besoins + faisabilité informatique
- **Résultat :** Spécification Technique de Besoin (STB)
  - → **ce que fait** le logiciel, mais **pas comment**
- **Remarques :**
  - → pas de choix d'implémentation
  - →(parfois) un manuel de référence préliminaire

17

Conception

- **But :**
  - → décrire **comment** le logiciel est construit
  - → décider **comment** utiliser la technologie pour répondre aux besoins
- **Travail :**
  - → enrichir la description de **détails d'implémentation**
  - → pour aboutir à une description **très proche** d'un programme
- **2 étapes :**
  - → **conception architecturale**
  - → **conception détaillée**

18

Conception architecturale

- **But :** décomposer le logiciel en composants
  - → mettre au point l'**architecture** du système
  - → définir les **sous-systèmes** et leurs **interactions**
  - → concevoir les **interfaces** entre composants
- **Résultat :**
  - → **description** de l'architecture globale du logiciel
  - → **spécification** des divers composants

19

Conception détaillée

- **But :** Élaborer les éléments internes de chaque sous-syst.
- **Résultat :**
  - → pour **chaque composant**, description des **structures de données**, **algorithmes**
- **Remarque :**
  - → si la **conception** est **possible**, la **faisabilité** est **démontrée**
  - → sinon, la **spécification** est **remise en cause**

Programmation

- **But :**
  → passer des structures de données et algorithmes à un **ensemble de programmes**

- **Résultat :**
  → ensemble de **programmes**
  → ensemble de **bibliothèques** / **modules**
  → **documentés** (commentaires)

- **Remarque :**
  → activité la mieux **maîtrisée** et **outillée** (parfois automatisée)

Gestion de configurations et Intégration

- **Gestion de configurations :**
  → **gérer les composants** logiciels (programmes, bibliothèques, …)
  → **maîtriser leur évolution** et leurs mises à jour

- **Intégration :**
  → **assembler** les composants pour obtenir un système exécutable

Vérification

- **But :** vérifier par rapport aux spécifications
  - → **vérifier** que les descriptions successives (et *en fin* le logiciel) **satisfont la STB**
- **Moyens :** revues de projet, tests
  - → test = **chercher des erreurs** dans un programme
  - → exécution sur un sous-ensemble fini de données
- **4 types de tests :**
  - → **test unitaire** : composants isolés
  - → **test d'intégration** : composants assemblés
  - → **test système** : système installé sur site
  - → **test d'acceptation** : testé par l'utilisateur

Validation

- **But :** vérifier par rapport aux utilisateurs
- **Moyen :** revues de projet

Maintenance

- **But :**
  - → **corriger** des défauts
  - → **améliorer** certaines caractéristiques
  - → **suivre les évolutions** (besoin, environnement)
- **Remarque :**
  - → peut remettre en cause les fonctions du système
  - → peut impliquer un re-développement *(re-ingeneering)*